

All about Data Engineering



PYSPARK POWER GUIDE



What is PySpark ?

PySpark is nothing but the **Python API** for Apache Spark.

It offers **PySpark Shell** which connects the *Python API* to the spark core and in turn initializes the **Spark context**.

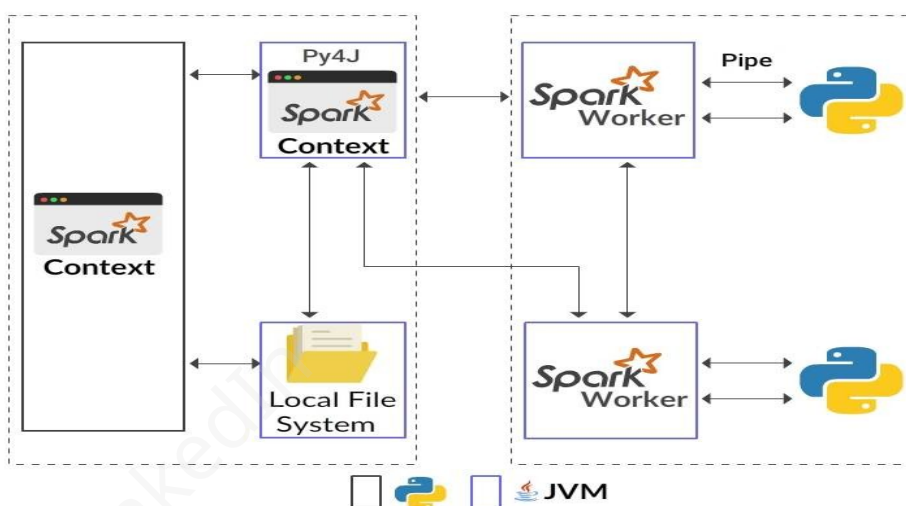
More on PySpark

- For any spark functionality, the entry point is **SparkContext**.
- SparkContext uses **Py4J** to launch a JVM and creates a **JavaSparkContext**.
- By default, PySpark has SparkContext available as **sc**, so creating a new SparkContext won't work.

Py4J

- PySpark is built on top of **Spark's Java API**.
- Data is processed in Python and **cached / shuffled** in the JVM.
- Py4J enables Python programs running in a **Python interpreter** to dynamically access **Java objects in a Java Virtual Machine**.
- Here methods are called as if the Java objects resided in the **Python interpreter and Java collections**. can be accessed through standard **Python collection methods**.

More on Py4J



- In the Python driver program, **SparkContext** uses **Py4J** to launch a **JVM** and create a **JavaSparkContext**.

- To establish local communication between the **Python and Java SparkContext objects** Py4J is used on the driver.

Installing and Configuring PySpark

- PySpark requires **Python 2.6 or higher**.
- PySpark applications are executed using a standard **CPython** interpreter in order to support **Python modules that use C extensions**.
- By default, PySpark requires python to be available on the **system PATH** and use it to run programs.
- Among PySpark's library dependencies all of them are bundled with **PySpark** including Py4J and they are **automatically imported**.

Getting Started

We can enter the `Spark's python environment` by running the given command in the shell.

```
./bin/pyspark
```

This will start your `PySpark shell`.

Python 2.7.12 (default, Nov 20 2017, 18:23:56)

```
[GCC 5.4.0 20160609] on linux2
```

Type "help", "copyright", "credits" or "license" for more information.

Welcome to

```

      _____
     /  _/  _  _  _  _/  /  _
    _\  \  _  \  _  \  _/  _/
 /  _/  /  .  _/  \  _/  /  _/  \  \    version 2.2.0
    /  /

```

Using Python version 2.7.12 (default, Nov 20 2017 18:23:56)

```
SparkSession available as 'spark'.
```

<<<

Resilient Distributed Datasets (RDDs)

- **Resilient distributed datasets (RDDs)** are known as the main abstraction in Spark.
- It is a partitioned collection of objects spread across a cluster, and can be persisted in memory or on disk.
- Once RDDs are created they are immutable.

There are two ways to create RDDs:

1. Parallelizing a collection in **driver program**.
2. Referencing one dataset in an external storage system, like a shared **filesystem**, **HBase**, **HDFS**, or any data source providing a **Hadoop InputFormat**.

Features Of RDDs

- **Resilient**, i.e. tolerant to faults using **RDD lineage graph** and therefore ready to recompute **damaged** or **missing** partitions due to **node failures**.
- **Dataset** - A set of partitioned data with primitive values or values of values, For example, **records or tuples**.
- **Distributed** with data remaining on **multiple nodes** in a cluster.

Creating RDDs

Parallelizing a collection in driver program.

E.g., here is how to create a **parallelized** collection holding the numbers 1 to 5:

```
data = [1, 2, 3, 4, 5]
distData = sc.parallelize(data)
```

Here, **distData** is the new **RDD** created by calling **SparkContext's parallelize** method.

Creating RDDs

Referencing one dataset in an external storage system, like a **shared filesystem**, **HBase**, **HDFS**, or any data source providing a **Hadoop InputFormat**.

For example, text file RDDs can be created using the method **SparkContext's textFile**.

For the file (local path on the machine, hdfs://, s3n://, etc URI) the above method takes a URI and then reads it as a collection containing lines to produce the RDD.

```
distFile = sc.textFile("data.txt")
```

RDD Operations

RDDs support two types of operations: **transformations**, which create a new dataset from an existing one, and **actions**, which return a value to the driver program after running a computation on the dataset.

For example, **map** is a **transformation** that passes each dataset element through a function and returns a new RDD representing the results.

Similarly, **reduce** is an **action** which aggregates all RDD elements by using some functions and then returns the final result to driver program.

More On RDD Operations

As a recap to RDD basics, consider the simple program shown below:

```
lines = sc.textFile("data.txt")
lineLengths = lines.map(lambda s: len(s))
totalLength = lineLengths.reduce(lambda a, b: a + b)
```

The first line defines a **base RDD** from an **external file**.

The second line defines **lineLengths** as the result of a **map transformation**.

Finally, in the third line, we run **reduce**, which is an **action**.

Transformations

- Transformations are functions that use an RDD as the input and return one or more RDDs as the output.
- **randomSplit**, **cogroup**, **join**, **reduceByKey**, **filter**, and **map** are examples of few transformations.
- Transformations do not change the input RDD, but always create one or more new RDDs by utilizing the computations they represent.
- By using transformations, you incrementally create an RDD lineage with all the parent RDDs of the last RDD.
- Transformations are *lazy*, i.e. are not run immediately. Transformations are done on demand.
- Transformations are executed only after calling an action.

Examples Of Transformations

- **filter(func)**: Returns a new dataset (RDD) that are created by choosing the elements of the source on which the function returns true.

- **map(func):** Passes each element of the RDD via the supplied function.
- **union():** New RDD contains elements from source argument and RDD.
- **intersection():** New RDD includes only common elements from source argument and RDD.
- **cartesian():** New RDD cross product of all elements from source argument and RDD.

Actions

- Actions return concluding results of RDD computations.
- Actions trigger execution utilising lineage graph to load the data into original RDD, and then execute all intermediate transformations and write final results out to file system or return it to Driver program.
- *Count, collect, reduce, take, and first* are few actions in spark.

Example of Actions

- **count():** Get the number of data elements in the RDD.
- **collect():** Get all the data elements in an RDD as an array.
- **reduce(func):** Aggregate the data elements in an RDD using this function which takes two arguments and returns one.
- **take (n):** Fetch first n data elements in an RDD computed by driver program.
- **foreach(func):** Execute function for each data element in RDD. usually used to update an accumulator or interacting with external systems.
- **first():** Retrieves the first data element in RDD. It is similar to take(1).
- **saveAsTextFile(path):** Writes the content of RDD to a text file or a set of text files to local file system/HDFS.

What is Dataframe ?

In general **DataFrames** can be defined as a data structure, which is tabular in nature. It represents rows, each of them consists of a number of observations.

Rows can have a variety of data formats (*heterogeneous*), whereas a column can have data of the same data type (*homogeneous*).

They mainly contain some *metadata* in addition to data like column and row names.

Why DataFrames ?

- DataFrames are widely used for processing a large collection of structured or semi-structured data
- They are having the ability to handle petabytes of data
- In addition, it supports a wide range of data format for reading as well as writing

As a conclusion DataFrame is data organized into named columns

Features of DataFrame

- Distributed
- Lazy Evals
- Immutable

Features Explained

- DataFrames are **Distributed** in Nature, which makes it fault tolerant and highly available data structure.
- **Lazy Evaluation** is an evaluation strategy which will hold the evaluation of an expression until its value is needed.
- DataFrames are **Immutable** in nature which means that it is an object whose state cannot be modified after it is created.

DataFrame Sources

For constructing a DataFrame a wide range of sources are available such as:

- Structured data files
- Tables in Hive
- External Databases
- Existing RDDs

Spark SQL

Spark introduces a programming module for structured data processing called **Spark SQL**.

It provides a programming abstraction called *DataFrame* and can act as distributed **SQL query engine**.

Features of Spark SQL

The main capabilities of using structured and semi-structured data, by **Spark SQL**. Such as:

- Provides DataFrame abstraction in Scala, Java, and Python.
- Spark SQL can read and write data from Hive Tables, JSON, and Parquet in various structured formats.
- Data can be queried by using Spark SQL.

For more details about Spark SQL refer the fresco course [Spark SQL](#)

Important classes of Spark SQL and DataFrames

- `pyspark.sql.Session` : Main entry point for Dataframe SparkSQL functionality
- `pyspark.sql.DataFrame` : A distributed collection of data grouped into named columns
- `pyspark.sql.Column` : A column expression in a DataFrame.
- `pyspark.sql.Row` : A row of data in a DataFrame.
- `pyspark.sql.GroupedData` : Aggregation methods, returned by `DataFrame.groupBy()`.

More On Classes

- `pyspark.sql.DataFrameNaFunctions` : Methods for handling missing data (null values).
- `pyspark.sql.DataFrameStatFunctions` : Methods for statistics functionality.
- `pyspark.sql.functions` : List of built-in functions available for DataFrame.
- `pyspark.sql.types` : List of data types available.
- `pyspark.sql.Window` : For working with window functions.

Creating a DataFrame demo

The entry point into all functionality in Spark is the **SparkSession** class.

To create a basic `SparkSession`, just use `SparkSession.builder`:

```
from pyspark.sql import SparkSession
spark = SparkSession \
    .builder \
    .appName("Data Frame Example") \
    .config("spark.some.config.option", "some-value") \
    .getOrCreate()
```

More On Creation

Import the `sql module` from `pyspark`

```
from pyspark.sql import *
```

```

Student = Row("firstName", "lastName", "age", "telephone")
s1 = Student('David', 'Julian', 22, 100000)
s2 = Student('Mark', 'Webb', 23, 658545)
StudentData=[s1,s2]
df=spark.createDataFrame(StudentData)
df.show()

```

```

from pyspark.sql import SparkSession
from pyspark import *
spark = SparkSession \
    .builder \
    .config("spark.some.config.option", "some-value") \
    .getOrCreate()
passenger = Row("Name", "age", "source", "destination")
s1 = passenger('David', 22, 'London', 'Paris')
s2 = passenger('Steve', 22, 'New York', 'Sydney')
x = [s1,s2]
df1=spark.createDataFrame(x)
df1.show()

```

Result of show()

Once `show()` is executed we can view the following result in the `pyspark shell`

```

+-----+-----+---+-----+
|firstName|lastName|age|telephone|
+-----+-----+---+-----+
|   David|   Julian| 22|   100000|
|   Mark|   Webb| 23|   658545|

```

Data Sources

- Spark SQL supports operating on a variety of `data sources` through the DataFrame interface.
- A DataFrame can be operated on using `relational transformations` and can also be used to create a temporary view.
- Registering a DataFrame as a temporary view allows you to run `SQL queries` over its data.

- This chapter describes the general methods for **loading and saving data** using the Spark Data Sources.

Generic Load/Save Functions

In most of the cases, the default data source will be used for all operations.

```
df = spark.read.load("file path")

# Spark load the data source from the defined file path

df.select("column name", "column name").write.save("file name")

# The DataFrame is saved in the defined format

# By default it is saved in the Spark Warehouse
```

File path can be from *local machine* as well as from *HDFS*.

Manually Specifying Options

You can also manually specify the **data source** that will be used along with any **extra options** that you would like to pass to the data source.

Data sources fully qualified name is used to specify them, but for built-in sources, you can also use their short names (**json, parquet, jdbc, orc, libsvm, csv, text**).

Specific File Formats

DataFrames which are loaded from any type of data can be converted to other types by using the syntax shown below.

A json file can be loaded:

```
df = spark.read.load("path of json file", format="json")
```

Apache Parquet

- Apache Parquet is a **columnar storage format** available to all projects in the **Hadoop ecosystem**, irrespective of the choice of the framework used for data processing, the model of data or programming language used.
- Spark SQL provides support for both **reading and writing Parquet files**.
- Automatic conversion to **nullable** occurs when one tries to write Parquet files, This is done due to **compatibility reasons**.

Reading A Parquet File

Here we are loading a `json` file into a dataframe.

```
df = spark.read.json("path of the file")
```

For saving the dataframe into `parquet` format.

```
df.write.parquet("parquet file name")
```

```
# Put your code here
from pyspark.sql import *
spark = SparkSession.builder.getOrCreate()
df = spark.read.json("emp.json")
df.show()
df.write.parquet("Employees")
df.createOrReplaceTempView("data")
res = spark.sql("select age,name,stream from data where stream='JAVA'")
res.show()
res.write.parquet("JavaEmployees")
```

Verifying The Result

We can verify the result by loading in `Parquet` format.

```
pf = spark.read.parquet("parquet file name")
```

Here we are reading in `Parquet` format.

To view the the `DataFrame` use `show()` method.

Why Parquet File Format ?

- Parquet stores nested data structures in a `flat columnar format`.
- On comparing with the traditional way instead of storing in `row-oriented way` in parquet is more `efficient`
- Parquet is the choice of `Big data` because it serves both needs, `efficient` and `performance` in both `storage` and `processing`.

Why Parquet File Format ?

- Parquet stores nested data structures in a `flat columnar format`.
- On comparing with the traditional way instead of storing in `row-oriented way` in parquet is more `efficient`

- Parquet is the choice of **Big data** because it serves both needs, **efficient** and **performance** in both **storage** and **processing**.

Advanced Concepts in Data Frame

In this chapter, you will learn how to perform some advanced operations on **DataFrames**.

Throughout the chapter, we will be focusing on **csv** files.

Reading Data From A CSV File

What is a CSV file?

- **CSV** is a file format which allows the user to store the data in tabular format.
- CSV stands for **comma-separated values**.
- It's data fields are most often **separated**, or **delimited**, by a **comma**.

CSV Loading

To load a **csv** data set user has to make use of **spark.read.csv** method to load it into a DataFrame.

Here we are loading a **football player dataset** using the spark **csvreader**.

```
df = spark.read.csv("path-of-file/fifa_players.csv", inferSchema = True, header = True)
```

CSV Loading

inferSchema (default false): From the data, it infers the input schema automatically.

header (default false): Using this it inherits the first line as column names.

To verify we can run **df.show(2)**.

The argument **2** will display the first two rows of the resulting DataFrame.

For every example from now onwards we will be using football player DataFrame

Schema of DataFrame

What is meant by **schema**?

It's just the **structure** of the DataFrame.

To check the schema one can make use of **printSchema** method.

It results in different **columns** in our DataFrame, along with the **datatype** and the **nullable** conditions.

How To Check The Schema

To check the **schema** of the loaded **csv** data.

```
df.printSchema()
```

Once executed we will get the following result.

```
root
|-- ID: integer (nullable = true)
|-- Name: string (nullable = true)
|-- Age: integer (nullable = true)
|-- Nationality: string (nullable = true)
|-- Overall: integer (nullable = true)
|-- Potential: integer (nullable = true)
|-- Club: string (nullable = true)
|-- Value: string (nullable = true)
|-- Wage: string (nullable = true)
|-- Special: integer (nullable = true)
```

Column Names and Count (Rows and Column)

For finding the column names, count of the number of rows and columns we can use the following methods.

For Column names

```
df.columns
['ID', 'Name', 'Age', 'Nationality', 'Overall', 'Potential', 'Club', 'Value', 'Wage', 'Special']
```

Row count

```
df.count()
17981
```

Column count

```
len(df.columns)
10
```

Describing a Particular Column

To get the **summary** of any particular column make use of **describe** method.

This method gives us the **statistical summary** of the given **column**, if not specified, it provides the **statistical summary** of the **DataFrame**.

```
df.describe('Name').show()
```

The result we will be as shown below.

```
+-----+-----+
|summary|      Name|
+-----+-----+
|  count|    17981|
|   mean|     null|
| stddev|     null|
|   min|    A. Abbas|
|   max|Óscar Whalley|
+-----+-----+
```

Advanced Concepts in Data Frame

In this chapter, you will learn how to perform some advanced operations on **DataFrames**.

Throughout the chapter, we will be focusing on **csv** files.

Reading Data From A CSV File

What is a CSV file?

- **CSV** is a file format which allows the user to store the data in tabular format.
- CSV stands for **comma-separated values**.
- It's data fields are most often **separated**, or **delimited**, by a **comma**.

CSV Loading

To load a **csv** data set user has to make use of **spark.read.csv** method to load it into a DataFrame.

Here we are loading a **football player dataset** using the spark `csvreader`.

```
df = spark.read.csv("path-of-file/fifa_players.csv", inferSchema = True, header = True)
```

CSV Loading

inferSchema (default false): From the data, it infers the input schema automatically.

header (default false): Using this it inherits the first line as column names.

To verify we can run `df.show(2)`.

The argument `2` will display the first two rows of the resulting DataFrame.

For every example from now onwards we will be using football player DataFrame

Schema of DataFrame

What is meant by **schema**?

It's just the **structure** of the DataFrame.

To check the schema one can make use of `printSchema` method.

It results in different **columns** in our DataFrame, along with the **datatype** and the **nullable** conditions.

How To Check The Schema

To check the **schema** of the loaded **csv** data.

```
df.printSchema()
```

Once executed we will get the following result.

```
root
|-- ID: integer (nullable = true)
|-- Name: string (nullable = true)
|-- Age: integer (nullable = true)
|-- Nationality: string (nullable = true)
|-- Overall: integer (nullable = true)
|-- Potential: integer (nullable = true)
|-- Club: string (nullable = true)
|-- Value: string (nullable = true)
|-- Wage: string (nullable = true)
```

```
|-- Special: integer (nullable = true)
```

Column Names and Count (Rows and Column)

For finding the column names, count of the number of rows and columns we can use the following methods.

For Column names

```
df.columns  
['ID', 'Name', 'Age', 'Nationality', 'Overall', 'Potential', 'Club', 'Value',  
'Wage', 'Special']
```

Row count

```
df.count()  
17981
```

Column count

```
len(df.columns)  
10
```

Describing a Particular Column

To get the **summary** of any particular column make use of **describe** method.

This method gives us the **statistical summary** of the given **column**, if not specified, it provides the **statistical summary** of the **DataFrame**.

```
df.describe('Name').show()
```

The result we will be as shown below.

```
+-----+-----+  
|summary|      Name|  
+-----+-----+  
|  count|    17981|  
|   mean|      null|  
| stddev|      null|  
|   min|    A. Abbas|  
|   max|Oscar Whalley|
```

```
+-----+-----+
```

Describing A Different Column

Now try it on some other column.

```
df.describe('Age').show()
```

Observe the various **Statistical Parameters**.

```
+-----+-----+
|summary|      Age|
+-----+-----+
|  count|      17981|
|   mean|25.144541460430453|
|  stddev| 4.614272345005111|
|    min|           16|
|    max|           47|
+-----+-----+
```

Selecting Multiple Columns

For selecting particular columns from the DataFrame, one can use the select method.

Syntax for performing selection operation is:

```
df.select('Column name 1','Column name 2',.....,'Column name n').show()
```

****show() is optional ****

One can load the result into another DataFrame by simply equating.

ie

```
dfnew=df.select('Column name 1','Column name 2',.....,'Column name n')
```

Selection Operation

Selecting the column **ID** and **Name** and loading the result to a new **DataFrame**.

```
dfnew=df.select('ID','Name')
```

Verifying the result

```
dfnew.show(5)
```

```
+-----+-----+
|    ID|          Name|
+-----+-----+
| 20801|Cristiano Ronaldo|
|158023|          L. Messi|
|190871|          Neymar|
|176580|          L. Suárez|
|167495|          M. Neuer|
+-----+-----+
```

only showing top 5 rows

Filtering Data

For filtering the data `filter` command is used.

```
df.filter(df.Club=='FC Barcelona').show(3)
```

The result will be as follows:

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+
|    ID|      Name|Age|Nationality|Overall|Potential|      Club| Value| Wage|
|Special|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|158023|  L. Messi| 30|  Argentina|    93|    93|FC Barcelona| €105M|€565K|
2154|
|176580| L. Suárez| 30|   Uruguay|    92|    92|FC Barcelona|  €97M|€510K|
2291|
|168651|I. Rakitić| 29|   Croatia|    87|    87|FC Barcelona|€48.5M|€275K|
2129|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+
```

only showing top 3 rows since we had given 3 in the show() as the argument

Verify the same by your own

To Filter our data based on multiple conditions (AND or OR)

```
df.filter((df.Club=='FC Barcelona') & (df.Nationality=='Spain')).show(3)
```

ID	Name	Age	Nationality	Overall	Potential	Club	Value
152729	Piqué	30	Spain	87	87	FC Barcelona	€37.5M
41	Iniesta	33	Spain	87	87	FC Barcelona	€29.5M
189511	Sergio Busquets	28	Spain	86	86	FC Barcelona	€36M

only showing top 3 rows

In a similar way we can use other [logical operators](#).

Sorting Data (OrderBy)

To sort the data use the [OrderBy](#) method.

In pyspark in default, it will sort in [ascending](#) order but we can change it into [descending](#) order as well.

```
df.filter((df.Club=='FC Barcelona') & (df.Nationality=='Spain')).orderBy('ID',).show(5)
```

To sort in descending order:

```
df.filter((df.Club=='FC Barcelona') & (df.Nationality=='Spain')).orderBy('ID',ascending=False).show(5)
```

Sorting

The result of the first order by operation results in the following output.

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+
```

ID	Name	Age	Nationality	Overall	Potential	Club	Value
41	Iniesta	33	Spain	87	87	FC Barcelona	€29.5M
152729	Piqué	30	Spain	87	87	FC Barcelona	€37.5M
189332	Jordi Alba	28	Spain	85	85	FC Barcelona	€30.5M
189511	Sergio Busquets	28	Spain	86	86	FC Barcelona	€36M
199564	Sergi Roberto	25	Spain	81	86	FC Barcelona	€19.5M

only showing top 5 rows

Random Data Generation

Random Data generation is useful when we want to test algorithms and to implement new ones.

In Spark under `sql.functions` we have methods to generate random data. e.g., uniform (`rand`), and standard normal (`randn`).

```
from pyspark.sql.functions import rand, randn
df = sqlContext.range(0, 7)
df.show()
```

The output will be as follows

```
+---+
| id |
+---+
|  0 |
|  1 |
|  2 |
|  3 |
+---+
```

More on Random Data

By using uniform distribution and normal distribution generate two more columns.

```
df.select("id", rand(seed=10).alias("uniform"),
randn(seed=27).alias("normal")).show()
```

```
+---+-----+-----+
| id|          uniform|          normal|
+---+-----+-----+
| 0|0.41371264720975787| 0.5888539012978773|
| 1| 0.1982919638208397|0.06157382353970104|
| 2|0.12030715258495939| 1.0854146699817222|
| 3|0.44292918521277047|-0.4798519469521663|
+---+-----+-----+
```

Summary and Descriptive Statistics

The **first operation** to perform after importing data is to get some **sense** of what it looks like.

The function **describe** returns a DataFrame containing information such as number of non-null entries (count), **mean**, **standard deviation**, and **minimum** and **maximum** value for each numerical column.

Summary and Descriptive Statistics

```
df.describe('uniform', 'normal').show()
```

The result is as follows:

```
+-----+-----+-----+
|summary|          uniform|          normal|
+-----+-----+-----+
| count|              10|              10|
| mean| 0.3841685645682706|-0.15825812884638607|
| stddev|0.31309395532409323| 0.963345903544872|
| min|0.03650707717266999|-2.1591956435415334|
| max| 0.8898784253886249| 1.0854146699817222|
+-----+-----+-----+
```

Descriptive Statistics

For a quick review of a column **describe** works fine.

In the same way, we can also make use of some standard **statistical functions** also.

```
from pyspark.sql.functions import mean, min, max
df.select([mean('uniform'), min('uniform'), max('uniform')]).show()
+-----+-----+-----+
|      avg(uniform)|      min(uniform)|      max(uniform)|
+-----+-----+-----+
|0.3841685645682706|0.03650707717266999|0.8898784253886249|
+-----+-----+-----+
```

Sample Co-Variance and Correlation

In statistics **Co-Variance** means how one **random variable** changes with respect to other.

Positive value indicates a trend in increase when the other increases.

Negative value indicates a trend in decrease when the other increases.

The sample co-variance of two columns of a DataFrame can be calculated as follows:

More On Co-Variance

```
from pyspark.sql.functions import rand
df = sqlContext.range(0, 10).withColumn('rand1',
rand(seed=10)).withColumn('rand2', rand(seed=27))
df.stat.cov('rand1', 'rand2')
0.031109767020625314
```

From the above we can infer that **co-variance** of two random columns is near to **zero**.

Correlation

Correlation provides the statistical dependence of two random variables.

```
df.stat.corr('rand1', 'rand2')
0.30842745432650953
```

Two randomly generated columns have **low correlation value**.

Cross Tabulation (Contingency Table)

Cross Tabulation provides a **frequency distribution table** for a given set of variables.

One of the powerful tool in statistics to observe the statistical **independence** of variables.

Consider an example

```
# Create a DataFrame with two columns (name, item)
names = ["Alice", "Bob", "Mike"]
items = ["milk", "bread", "butter", "apples", "oranges"]
df = sqlContext.createDataFrame([(names[i % 3], items[i % 5]) for i in
range(100)], ["name", "item"])
```

Contingency Table)

For applying the **cross tabulation** we can make use of the crosstab method.

```
df.stat.crosstab("name", "item").show()
+-----+-----+-----+-----+-----+
|name_item|apples|bread|butter|milk|oranges|
+-----+-----+-----+-----+-----+
|      Bob|      6|      7|      7|      6|      7|
|      Mike|      7|      6|      7|      7|      6|
|      Alice|      7|      7|      6|      7|      7|
+-----+-----+-----+-----+-----+
```

Cardinality of columns we run **crosstab** on cannot be too big.

```
# Put your code here
from pyspark.sql import *
from pyspark import SparkContext
from pyspark.sql.functions import rand, randn
from pyspark.sql import SQLContext
from pyspark.sql.types import FloatType
spark = SparkSession.builder.getOrCreate()
df1 = Row("1", "2")
sqlContext = SQLContext(spark)
df = sqlContext.range(0, 10).withColumn('rand1', rand(seed=10)).withColumn('rand2', rand(seed=27))
a = df.stat.cov('rand1', 'rand2')
b = df.stat.corr('rand1', 'rand2')

s1 = df1("Co-variance", a)
s2 = df1("Correlation", b)
x=[s1, s2]
```

```
df2 = spark.createDataFrame(x)
df2.show()
df2.write.parquet("Result")
```

What is Spark SQL ?

Spark SQL brings native support for **SQL** to Spark.

Spark SQL blurs the lines between **RDD's** and **relational tables**.

By integrating these powerful features Spark makes it easy for developers to use SQL commands for querying external data with complex analytics, all within in a single application.

Performing SQL Queries

We can also pass **SQL queries** directly to any DataFrame.

For that, we need to create a table from the DataFrame using the **registerTempTable** method.

After that use **sqlContext.sql()** to pass the SQL queries.

Apache Hive



The **Apache Hive** data warehouse software allows **reading**, **writing**, and managing **large datasets** residing in distributed storage and queried using **SQL syntax**.

Features of Apache Hive

Apache Hive is built on top of Apache Hadoop.

The below mentioned are the features of Apache Hive.

- Apache Hive is having tools to allow easy and quick access to data using SQL, thus enables data warehousing tasks such like extract/transform/load (ETL), reporting, and data analysis.
- Mechanisms for imposing structure on a variety of data formats.
- Access to files stored either directly in Apache HDFS or in other data storage systems such as Apache HBase.

Features Of Apache Hive

- Query execution via Apache Tez, Apache Spark, or MapReduce.
- A procedural language with HPL-SQL.
- Sub-second query retrieval via Hive LLAP, Apache YARN and Apache Slider.

What Hive Provides ?

Apache Hive provides the standard SQL functionalities, which includes many of the later SQL:2003 and SQL:2011 features for analytics.

We can extend Hive's SQL with the user code by using user-defined functions (UDFs), user-defined aggregates (UDAFs), and user-defined table functions (UDTFs).

Hive comes with built-in connectors for comma and tab-separated values (CSV/TSV) text files, Apache Parquet, Apache ORC, and other formats.

Spark SQL supports reading and writing data stored in Hive. Connecting Hive From Spark

When working with Hive, one must instantiate SparkSession with Hive support, including connectivity to a persistent Hive metastore, support for Hive serdes, and Hive user-defined functions.

How To Enable Hive Support

```
from os.path import expanduser, join, abspath

from pyspark.sql import SparkSession
from pyspark.sql import Row
```

```
#warehouse_location points to the default location for managed databases and tables
```

```
warehouse_location = abspath('spark-warehouse')
```

```
spark = SparkSession \
    .builder \
    .appName("Python Spark SQL Hive integration example") \
    .config("spark.sql.warehouse.dir", warehouse_location) \
    .enableHiveSupport() \
    .getOrCreate()
```

Creating Hive Table From Spark

We can easily create a table in **hive warehouse** programmatically from Spark.

The syntax for creating a table is as follows:

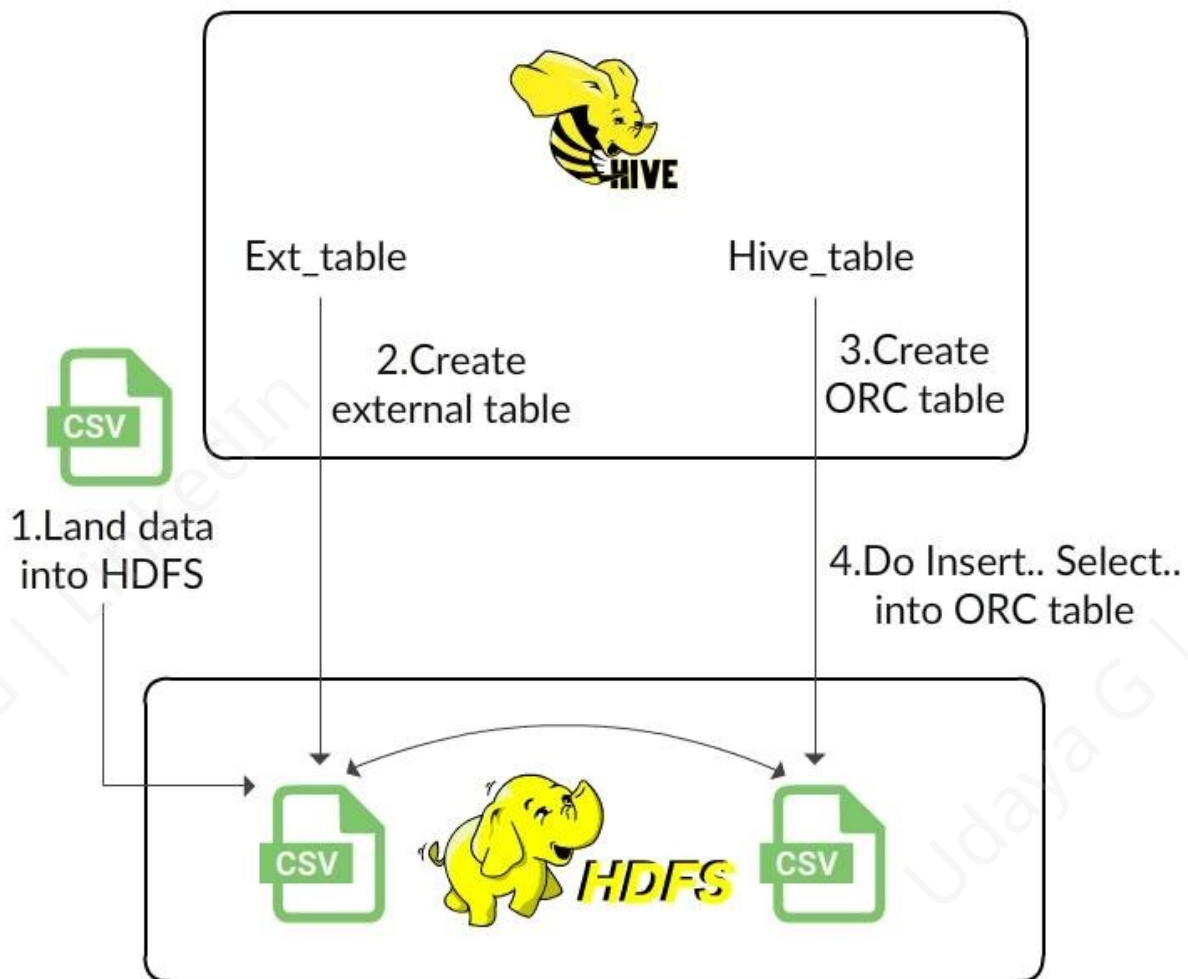
```
spark.sql("CREATE TABLE IF NOT EXISTS table_name(column_name_1  
DataType,column_name_2 DataType,.....,column_name_n DataType) USING hive")
```

To load a DataFrame into a table.

```
df.write.insertInto("table name",overwrite = True)
```

Now verify the result by using the select statement.

Hive External Table



- External tables are used to store data **outside** the hive.
- Data needs to remain in the **underlying location** even after the user drop the table.

Handling External Hive Tables From Apache Spark

First, create an **external table** in the hive by specifying the location.

One can create an external table in the hive by running the following query on **hive shell**.

```
hive> create external table table_name(column_name1 DataType,column_name2  
DataType,.....,column_name_n DataType) STORED AS Parquet location ' path of  
external table';
```

- The table is created in **Parquet schema**.
- The table is saved in the **hdfs** directory.

Loading Data From Spark To The Hive Table

We can load data to `hive table` from the `DataFrame`.

For doing the same `schema` of both `hive table` and the `DataFrame` should be equal.

Let us take a sample CSV file.

We can read the `csv` the file by making use of `spark csv reader`.

```
df = spark.read.csv("path-of-file", inferSchema = True, header = True)
```

The `schema` of the `DataFrame` will be same as the schema of the CSV file itself.

Data Loading To External Table

For loading the data we have to save the dataframe in external hive table location.

```
df.write.mode('overwrite').format("format").save("location")
```

Since our hive external table is in `parquet format` in place of format we have to mention 'parquet'.

The location should be `same` as the hive external table location in hdfs directory.

If the `schema` is matching then data will load `automatically` to the hive table.

By querying the hive table we can verify it.

What is HBase ?



HBase is a `distributed` column-oriented data store built on top of `HDFS`.

HBase is an Apache open source project whose goal is to provide **storage** for the Hadoop Distributed Computing.

Data is logically organized into **tables**, **rows** and **columns**.

More On HBase

- HBase features compression, in-memory operation, and Bloom filters on a per-column basis as outlined in the original **Bigtable** paper.
- Tables in HBase can serve as the **input** and **output** for Map Reduce jobs run in Hadoop, and may be accessed through the **Java API** but also through **REST**, **Avro** or **Thrift gateway APIs**.
- It is a column-oriented key-value data store and has been idolized widely because of its **lineage** with **Hadoop** and **HDFS**.
- HBase runs on **top** of HDFS and is well-suited for **faster** read and write operations on large datasets with high **throughput** and low **input/output latency**.

How To Connect Spark and HBase

- To connect we require **hdfs**, **Spark** and **HBase** installed in the local machine.
- Make sure that your **versions** are **matching** with each other.
- Copy all the **HBase jar** files to the **Spark lib** folder.
- Once done set the **SPARK_CLASSPATH** in **spark-env.sh** with **lib**.

Building A Real-Time Data Pipeline

API Service

Real Time Pipeline using HDFS,Spark and HBase

Various Stages

It has 4 main stages which includes:

- Transformation
- Cleaning
- Validation
- Writing of the data received from the various sources

Transformation And Cleaning

Data Transformation

- This is an entry point for the streaming application.
- Here the operations related to normalization of data are performed.
- Transformation of data can be performed by using built-in functions like map, filter, foreachRDD etc.

Data Cleaning

- During **preprocessing** cleaning is very important.
- In this stage, we can use **custom built libraries** for cleaning the data.

Validation And Writing

Data Validation

In this stage, we can **validate** the data with respect to some **standard validations** such as length, patterns and so on.

Writing

At last, data passed from the **previous three stages** is passed on to the **writing application** which simply writes this final set of data to **HBase** for further data analysis.

Spark In Real World

Uber – the online taxi company is an apt example for Spark. They are gathering terabytes of event data from its various users.

- Uses **Kafka, Spark Streaming, and HDFS**, to build a continuous **ETL pipeline**.
- Convert raw unstructured data into structured data as it is collected.
- Uses it further complex analytics and optimization of operations.

Spark In Real World

Pinterest – Uses a Spark ETL pipeline

- Leverages Spark Streaming to gain **immediate insight** into how users all over the world are **engaging** with **Pins** in real time.
- Can make more relevant recommendations as people navigate the site.
- Recommends related Pins.
- Determine which products to **buy**, or **destinations** to visit.

Spark In Real World

Conviva – 4 million video feeds per month.

- Conviva is using Spark for reducing the customer churn by **managing live video traffic** and **optimizing video streams**.
- They maintain a consistently smooth high-quality viewing experience.

Spark In Real World

Capital One – makes use of Spark and data science algorithms for a better understanding of its customers.

- Developing the next generation of financial products and services.
- Find attributes and patterns of increased probability for fraud.

Netflix – Movie recommendation engine from user data.

- User data is also used for content creation

```
# Put your code here
from pyspark.sql import *
spark = SparkSession.builder.getOrCreate()
df = Row("ID","Name","Age","AreaofInterest")
s1 = df("1","Jack",22,"Data Science")
s2 = df("2","Leo",21,"Data Analytics")
s3 = df("3","Luke",24,"Micro Services")
s4 = df("4","Mark",21,"Data Analytics")
x = [s1,s2,s3,s4]
df1 = spark.createDataFrame(x)
df3 = df1.describe("Age")
df3.show()
df3.write.parquet("Age")
df1.createOrReplaceTempView("data")
df4 = spark.sql("select ID,Name,Age from data order by ID desc")
df4.show()
df4.write.parquet("NameSorted")
```

How to Work with Apache Spark and Delta Lake? — Part 1

Working with Spark and Delta Lake (Inspired by: Data Engineering with Databricks Cook Book, Packt)

How to Work with Apache Spark and Delta Lake? Part I



Synopsis of Part 1:

Data Ingestion & Data Extraction

Data Transformation & Manipulation

Data Management with Delta Lake

1. Data Ingestion & Data Extraction

In this section, we will learn about data ingestion and data extraction:

Reading CSV

- **Reading CSV Files:** Use Apache Spark to read CSV files from various storage locations. Utilize `spark.read.csv` with options for delimiter, header, and schema.
- **Header and Schema Inference:** By setting `header=True`, Spark interprets the first line as column names. Use `inferSchema=True` to let Spark deduce the data types of columns.
- **Reading Multiple CSV Files:** You can read multiple CSV files using wildcards or a list of paths.
- **Handling Delimiters:** Customize the CSV reading process with options like `sep` for delimiter, `quote` for quoting, and `escape` for escaping characters.
- **Schema Definition:** Define a custom schema using `StructType` and `StructField` for more control over data types and nullability.

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

spark = SparkSession.builder.appName("CSV Example").getOrCreate()

# Read CSV with header and inferred schema
df = spark.read.csv("path/to/csvfile.csv", header=True, inferSchema=True)

# Define custom schema
schema = StructType([
    StructField("name", StringType(), True),
    StructField("age", IntegerType(), True)
])
df_custom_schema = spark.read.schema(schema).csv("path/to/csvfile.csv")
```

Reading JSON Data with Apache Spark

- **Single-Line JSON:** Use `spark.read.json` to parse single-line JSON files. Spark infers the schema and data types automatically.
- **Multi-Line JSON:** Handle multi-line JSON files by setting the `multiLine` option to `True`, allowing Spark to parse complex JSON structures.

- **Schema Inference and Definition:** Spark can infer the schema from JSON files, or you can define a custom schema for more control.
- **Reading Nested JSON:** Spark can read nested JSON structures and create DataFrame columns accordingly. Use dot notation to access nested fields.
- **Handling Missing Values:** Manage missing or null values in JSON data by using options like `dropFieldIfAllNull`.

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

# Read single-line JSON
df = spark.read.json("path/to/singleline.json")

# Read multi-line JSON
df multiline = spark.read.option("multiLine", True).json("path/to/multiline.json")

# Define custom schema for JSON
schema = StructType([
    StructField("id", IntegerType(), True),
    StructField("name", StringType(), True),
    StructField("attributes", StructType([
        StructField("age", IntegerType(), True),
        StructField("gender", StringType(), True)
    ]))
])
df_custom_schema = spark.read.schema(schema).json("path/to/jsonfile.json")
```

Reading Parquet Data with Apache Spark

- **Reading Parquet Files:** Use `spark.read.parquet` to read Parquet files, which offer efficient storage and fast query performance.
- **Automatic Partition Discovery:** Spark automatically discovers and reads partitioned Parquet files, allowing for efficient querying.
- **Writing Parquet Files:** Use `df.write.parquet` to write data to Parquet format. Parquet files support efficient columnar storage and compression.
- **Compression Options:** Specify compression codecs such as `snappy`, `gzip`, or `lzo` to optimize storage space and read performance.
- **Schema Evolution:** Parquet supports schema evolution, allowing you to add or remove columns without breaking existing data.

```
# Read Parquet files
df = spark.read.parquet("path/to/parquetfile.parquet")

# Write Parquet files with compression
df.write.option("compression", "snappy").parquet("path/to/output_parquet/")
```

Parsing XML Data with Apache Spark

- **Reading XML Files:** Use the `spark-xml` library to parse XML files. Configure row tags to identify elements within the XML structure.
- **Handling Nested XML:** Parse nested XML structures into DataFrame columns. Define nested schemas using `StructType`.
- **Attribute and Value Tags:** Customize attribute prefixes and value tags to handle XML attributes and element values effectively.
- **Complex XML Parsing:** Manage complex XML files with multiple nested elements and attributes using advanced configurations.
- **Performance Considerations:** Optimize XML parsing performance by configuring parallelism and memory settings.

```
# Add the spark-xml library
df = spark.read.format("xml").option("rowTag", "book").load("path/to/xmlfile.xml")
```

Working with Nested Data Structures

- **Struct and Array Types:** Manage complex data types like structs and arrays. Use DataFrame API functions such as `select`, `withColumn`, and `explode` to manipulate nested data.
- **Accessing Nested Fields:** Use dot notation to access nested fields within structs. Apply transformations directly on nested columns.
- **Exploding Arrays:** Use the `explode` function to convert array elements into individual rows. This is useful for flattening nested arrays.

- **Creating Structs:** Use the `struct` function to create new struct columns from existing columns.
- **Handling Null Values:** Manage null values within nested structures using functions like `fillna`, `dropna`, and `isNull`.

```
from pyspark.sql.functions import explode, struct

# Explode nested array
df exploded = df.select("name", explode("attributes").alias("attribute"))

# Access nested fields
df_nested = df.select("name", "attributes.age", "attributes.gender")

# Create new struct
df_struct = df.withColumn("new_struct", struct("col1", "col2"))
```

2. Data Transformation & Manipulation

In this section, you will learn about the transformation and manipulation.

Transformations

- **Basic Transformations:** Apply transformations like `map`, `flatMap`, and `filter` to process data. Use DataFrame API for efficient row and column operations.
- **Column Manipulation:** Add, drop, or rename columns using `withColumn`, `drop`, and `withColumnRenamed` methods.
- **Conditional Transformations:** Use `when` and `otherwise` functions to apply conditional logic to DataFrame columns.
- **String Operations:** Perform string operations such as `concat`, `substring`, and `upper` on DataFrame columns.
- **Mathematical Operations:** Use mathematical functions like `abs`, `round`, and `sqrt` for numeric column transformations.
- **Date and Time Functions:** Apply date and time functions like `current_date`, `date_add`, and `datediff` to handle temporal data.

```

from pyspark.sql.functions import col, when, concat, lit

# Add a new column
df = df.withColumn("new_col", col("existing_col") * 2)

# Conditional transformation
df = df.withColumn("status", when(col("age") > 18, "Adult").otherwise("Minor"))

# String concatenation
df = df.withColumn("full_name", concat(col("first_name"), lit(" "), col("last_name")))

```

Filtering Data

- **Basic Filtering:** Filter DataFrame rows using the `filter` and `where` methods. Apply conditions directly or use SQL expressions.
- **Complex Conditions:** Combine multiple conditions using logical operators like `&`, `|`, and `~`.
- **Null Handling:** Filter rows based on the presence of null values using `isNull` and `isNotNull` functions.
- **String Matching:** Use string functions like `contains`, `startswith`, and `endswith` for filtering rows based on string patterns.
- **Range Filtering:** Apply range-based filtering using conditions like `between` and `isin`.

```

# Filter rows with age greater than 18
df_filtered = df.filter(col("age") > 18)

# Complex conditions
df_filtered = df.filter((col("age") > 18) & (col("gender") == "Male"))

# Null handling
df_filtered = df.filter(col("address").isNotNull())

```

Performing Joins

- **Basic Joins:** Join multiple DataFrames using `join` the method. Specify join conditions and join types like `inner`, `left`, `right`, and `outer`.

- **Broadcast Joins:** Optimize join performance with broadcast joins. Use `broadcast` function to broadcast smaller DataFrames.
- **Handling Duplicate Columns:** Resolve column name conflicts by renaming or selecting specific columns.
- **SQL Joins:** Perform joins using Spark SQL with `JOIN` clauses. Write SQL queries to join tables based on common columns.
- **Multi-way Joins:** Combine more than two DataFrames in a single join operation. Chain multiple `join` methods together.

```
from pyspark.sql.functions import broadcast

# Inner join
df_joined = df1.join(df2, df1.id == df2.id, "inner")

# Broadcast join
df_broadcast = df1.join(broadcast(df2), df1.id == df2.id, "inner")
```

Aggregations

- **Group By Aggregations:** Perform aggregations using `groupBy` and `agg` methods. Calculate sums, averages, counts, and other aggregate functions.
- **Pivot Tables:** Create pivot tables using `pivot` method to transform row data into columns.
- **Window Functions:** Apply window functions for operations like ranking, cumulative sums, and moving averages. Define window specifications with `partitionBy` and `orderBy`.
- **Custom Aggregations:** Implement custom aggregation logic using `udaf` for complex aggregation scenarios.
- **DataFrame Aggregations:** Use DataFrame-level aggregation methods like `agg`, `sum`, `avg`, and `count` for quick calculations.

```

from pyspark.sql import functions as F

# Group by and aggregation
df_grouped = df.groupBy("category").agg(F.sum("sales"), F.avg("sales"))

# Pivot table
df_pivot = df.groupBy("year").pivot("month").sum("sales")

```

Window functions with Apache Spark

- **Defining Windows:** Use `Window` specification to define partitioning and ordering for window functions. Apply window functions to calculate running totals, ranks, and moving averages.
- **Ranking Functions:** Use ranking functions like `row_number`, `rank`, and `dense_rank` to assign ranks to rows within partitions.
- **Aggregate Functions:** Apply aggregate functions within windows using `sum`, `avg`, `min`, `max`, and `count`.
- **Lead and Lag Functions:** Use `lead` and `lag` functions to access subsequent and previous rows within partitions.
- **Row and Range Windows:** Define windows based on row numbers or value ranges using `rowsBetween` and `rangeBetween`.

```

from pyspark.sql.window import Window
from pyspark.sql.functions import row_number, rank, sum

# Define window specification
windowSpec = Window.partitionBy("category").orderBy("date")

# Ranking functions
df = df.withColumn("rank", rank().over(windowSpec))

# Aggregate functions
df = df.withColumn("cumulative_sum", sum("sales").over(windowSpec))

```

Custom UDFs in Apache Spark

- **Creating UDFs:** Create User-Defined Functions (UDFs) to extend Spark's functionality. Register UDFs and apply them to DataFrame columns.

- **Scalar UDFs:** Define scalar UDFs that operate on individual rows. Register UDFs using `udf` function and apply them using `withColumn`.
- **Pandas UDFs:** Use Pandas UDFs for better performance with vectorized operations. Define UDFs with `pandas_udf` decorator.
- **Type Safety:** Ensure type safety by specifying input and output data types for UDFs.
- **Reusability:** Create reusable UDFs for common transformations and apply them across multiple DataFrames.

```
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

# Create a scalar UDF
def to_upper(s):
    return s.upper()

to_upper_udf = udf(to_upper, StringType())

# Apply the UDF
df = df.withColumn("upper_name", to_upper_udf(col("name")))
```

Handling Null Values with Spark

- **Fill Null Values:** Use `fillna` method to replace null values with specified values or default values for columns.
- **Drop Null Values:** Remove rows with null values using `dropna` method. Specify conditions to drop rows based on certain columns.
- **Replace Null Values:** Use `na.replace` method to replace null values with specified values across DataFrame.
- **Null Checks:** Check for null values in DataFrame columns using `isNull` and `isNotNull` functions.
- **Conditional Null Handling:** Apply conditional logic to handle null values using `when` and `otherwise` functions.

```
# Fill null values
df_filled = df.fillna({"age": 0, "name": "Unknown"})

# Drop rows with null values
df_dropped = df.dropna()

# Replace null values
df_replaced = df.na.replace({"": "Unknown", None: "Unknown"})
```

3. Data Management with Delta Lake

In this section, let us learn about data management with Delta Lake.

Creating Delta Lake

- **Initialize Delta Lake:** Create Delta tables from existing data using `spark.sql("CREATE TABLE ...")`. Delta Lake provides ACID transactions and scalable metadata handling.
- **Table Creation:** Use SQL `CREATE TABLE` syntax to define Delta tables. Specify storage locations and Delta-specific configurations.
- **Data Loading:** Load data into Delta tables using `df.write.format("delta").save()`. Delta Lake automatically tracks changes and manages transactional consistency.
- **Schema Management:** Manage schema evolution with Delta Lake. Add or modify columns without breaking existing data.
- **Partitioning:** Optimize data layout by partitioning Delta tables. Use `partitionBy` option to define partition columns for efficient querying.

```
# Create a Delta table from an existing DataFrame
df.write.format("delta").save("path/to/delta-table")

# Create a Delta table using SQL
spark.sql("""
CREATE TABLE delta_table
USING delta
LOCATION 'path/to/delta-table'
""")
```

Reading a Delta Lake

- **Read Delta Tables:** Use `spark.read.format("delta")` to read Delta tables. Delta Lake optimizes read performance with efficient data access.
- **Time Travel:** Access historical data versions using Delta Lake's time travel feature. Query-specific table snapshots by timestamp or version number.
- **Snapshot Isolation:** Delta Lake provides snapshot isolation, ensuring consistent reads even during concurrent writes.
- **Version History:** Use `DESCRIBE HISTORY` to track changes and view the version history of Delta tables.
- **Efficient Querying:** Leverage Delta Lake's indexing and caching mechanisms for efficient querying and fast data retrieval.

```
# Read a Delta table
df = spark.read.format("delta").load("path/to/delta-table")

# Time travel to a specific version
df version = spark.read.format("delta").option("versionAsOf", 2).load("path/to/delta-table")

# Describe history of a Delta table
spark.sql("DESCRIBE HISTORY delta_table")
```

Updating Data

- **Update Records:** Use Delta Lake's `update` method to modify records in Delta tables. Apply conditions to update specific rows based on criteria.
- **Conditional Updates:** Implement conditional logic in update statements to selectively update records.
- **Schema Enforcement:** Delta Lake enforces schema during updates, ensuring data consistency and integrity.
- **Transactional Updates:** Delta Lake provides ACID transactions, allowing you to update multiple records atomically.
- **Performance Optimization:** Optimize update performance by partitioning tables and leveraging Delta Lake's indexing mechanisms.

```

from delta.tables import DeltaTable

# Load a Delta table
deltaTable = DeltaTable.forPath(spark, "path/to/delta-table")

# Update records
deltaTable.update(
    condition = "id = 1",
    set = {"name": "'Updated Name'"}
)

```

Merging Data

- **Merge Statements:** Use Delta Lake's `merge` method to combine new data with existing Delta tables. Handle insert, update, and delete operations within a single transaction.
- **Conditional Merging:** Apply conditions in merge statements to selectively merge records based on specific criteria.
- **Handling Duplicates:** Use merge statements to deduplicate data and resolve conflicts.
- **Change Data Capture:** Implement change data capture (CDC) using merge statements to track and apply changes from source systems.
- **Performance Considerations:** Optimize merge performance by partitioning tables and using efficient indexing mechanisms.

```

# Merge new data into Delta table
deltaTable.merge(
    source = newData,
    condition = "deltaTable.id = newData.id",
    whenMatchedUpdate = {"deltaTable.name": "newData.name"},
    whenNotMatchedInsert = {"id": "newData.id", "name": "newData.name"}
)

```

Change Data Capture

- **Track Changes:** Use Delta Lake to track changes in Delta tables. Implement CDC to capture insertions, updates, and deletions.

- **Incremental Data Loading:** Load data incrementally using CDC, ensuring efficient and timely updates.
- **Data Consistency:** Ensure data consistency and integrity by capturing and applying changes accurately.
- **Historical Tracking:** Maintain a historical record of changes for auditing and analysis purposes.
- **Integration with ETL Pipelines:** Integrate CDC with ETL pipelines to automate data synchronization between systems.

```
# Capture changes in Delta table
changes = deltaTable.history()

# Load data incrementally
incrementalData = spark.read.format("delta").option("startingVersion",
"latest").load("path/to/delta-table")
```

Optimizing Delta Lake

- **Optimize Command:** Use the `OPTIMIZE` command to compact Delta table files. Improve query performance by reducing the number of small files.
- **Vacuum Command:** Remove old data files with the `VACUUM` command to reclaim storage space and manage table size.
- **Partitioning:** Optimize data layout by partitioning Delta tables based on query patterns. Use `partitionBy` to define partition columns.
- **Z-Ordering:** Use Z-Ordering to optimize data storage and retrieval. Z-Ordering improves query performance by co-locating related data.
- **Auto Optimize:** Enable Auto Optimize to automate file compaction and optimize table performance without manual intervention.

```
# Optimize Delta table
spark.sql("OPTIMIZE delta_table")
```

```
# Vacuum Delta table
spark.sql("VACUUM delta_table RETAIN 168 HOURS")
```

Versioning & Time Travel

- **Time Travel:** Access historical data versions using Delta Lake's time travel feature. Query specific table snapshots by timestamp or version number.
- **Version Control:** Delta Lake provides version control, allowing you to manage and revert to previous data versions.
- **Data Auditing:** Use time travel to audit changes and track the evolution of data over time.
- **Debugging:** Time travel helps in debugging and troubleshooting by providing access to past states of data.
- **Historical Analysis:** Perform historical analysis by querying previous versions of Delta tables.

```
# Time travel to a specific timestamp
df time travel = spark.read.format("delta").option("timestampAsOf", "2023-01-01").load("path/to/delta-table")

# Time travel to a specific version
df version = spark.read.format("delta").option("versionAsOf", 2).load("path/to/delta-table")
```

Managing Delta Lake

- **Table Management:** Manage Delta tables by configuring retention policies, schema evolution, and partitioning strategies.
- **Data Retention:** Configure data retention policies to manage the lifecycle of data. Use `VACUUM` the command to remove old data files.
- **Schema Evolution:** Delta Lake supports schema evolution, allowing you to add or modify columns without breaking existing data.

- **Partition Management:** Optimize table performance by managing partitions effectively. Use `partitionBy` to define partition columns.
- **Metadata Handling:** Delta Lake manages metadata efficiently, ensuring fast query performance and scalable table management.

```
# Configure retention policy
spark.sql("ALTER TABLE delta table SET TBLPROPERTIES ('delta.logRetentionDuration' = '30 days')")

# Manage schema evolution
df.write.option("mergeSchema", "true").format("delta").save("path/to/delta-table")
```

What's Next?

In the next section, we will explore the following topics:

- **Streaming Data:** Handle real-time data streams with Apache Spark. Set up data streams from sources like Kafka, socket connections, and files.
- **Processing Streaming Data:** Techniques for processing streaming data, including windowed operations and stateful processing. Manage watermarks and late data.
- **Performance Tuning:** Strategies for optimizing Spark applications, including resource allocation, partitioning, and shuffling. Use Spark UI for performance monitoring.

Synopsis of Part II:

Ingesting Streaming Data

Processing Streaming Data

1. Ingesting Streaming Data

In this section, let us learn different aspects and techniques involved in ingesting streaming data.

Configuring Spark Structured Streaming for Real-time Data

- **Define a Streaming Query:** Initiate a Spark session and define a streaming DataFrame to read data in real-time from sources like Kafka, socket, or file systems. Use `spark.readStream` to initiate this process.
- **Set Up Schema:** Explicitly define the schema of the incoming data to ensure that Spark can infer the structure of the data correctly and process it efficiently.
- **Configure Stream Triggers:** Use triggers to control the frequency at which streaming queries process data. Common options include fixed intervals or continuous processing for near real-time latency.
- **Handle Late Data with Watermarks:** Define watermarks to manage and filter out late data. Watermarks allow the system to retain a specified amount of data to account for lateness, ensuring accurate and complete results.
- **Checkpoints for Fault Tolerance:** Set up checkpoints to maintain state across failures. Use the `checkpointLocation` option to specify the directory for saving state information.
- **Stream Output Modes:** Choose the appropriate output mode for your use case — append, complete, or update. Each mode handles the output data differently based on the operation requirements.

- **Error Handling and Logging:** Implement robust error handling and logging mechanisms to monitor the health of the streaming application and troubleshoot issues in real-time.

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("StructuredStreamingExample").getOrCreate()
schema = "id INT, value STRING"

streaming_df = spark.readStream.format("kafka").option("kafka.bootstrap.servers",
"localhost:9092").option("subscribe", "topic").load().selectExpr("CAST(key AS
STRING)", "CAST(value AS STRING)")
streaming_df = streaming_df.selectExpr("CAST(value AS STRING)")

query =
streaming_df.writeStream.outputMode("append").format("console").option("checkpointLo
cation", "/path/to/checkpoint/dir").start()
query.awaitTermination()
```

Configuring Triggers for Structured Streaming in Apache Spark

- **Fixed-Interval Micro-batching:** Configure fixed-interval triggers to process data at regular intervals. This is useful for predictable workloads where data arrival is consistent.
- **Continuous Processing:** For low-latency applications, enable continuous processing mode. This provides microsecond-level latency but is limited to specific data sources and sinks.
- **One-Time Batch Processing:** Use one-time triggers to process the data available at the moment and stop the query. This is suitable for scenarios where data needs to be processed only once.
- **Custom Trigger Intervals:** Define custom intervals to optimize performance based on the specific requirements of your application. This can help balance the load and reduce processing latency.
- **Combining Triggers with Watermarks:** Utilize watermarks alongside triggers to manage late data effectively, ensuring that your application processes all relevant data within the specified time window.

- **Error and Retry Mechanisms:** Implement error handling and retry logic within your triggers to manage transient failures and ensure data consistency.
- **Monitoring and Tuning Triggers:** Continuously monitor the performance of your triggers and adjust the intervals based on the observed throughput and latency.

```
query =  
streaming_df.writeStream.outputMode("append").format("console").trigger(processingTime='10 seconds').option("checkpointLocation", "/path/to/checkpoint/dir").start()  
query.awaitTermination()
```

Reading Data from Real-time Sources, such as Apache Kafka, with Apache Spark Structured Streaming

- **Kafka Integration:** Leverage Spark's native Kafka integration to read streaming data directly from Kafka topics. Configure the Kafka broker addresses and topic names in the Spark `readStream` options.
- **Defining Schema for Kafka Data:** Explicitly define the schema for the Kafka data to ensure efficient processing. This involves parsing the key-value pairs into a structured format.
- **Handling Multiple Topics:** Configure Spark to read from multiple Kafka topics by specifying a comma-separated list of topic names. This is useful for applications that need to process data from various sources.
- **Offset Management:** Manage Kafka offsets to ensure that your streaming application processes each message exactly once. Use the `startingOffsets` option to specify the starting point.
- **Handling Data Format:** Decode and parse the Kafka message payload based on the data format (e.g., JSON, Avro) using appropriate Spark functions or third-party libraries.

- **Security Configurations:** Secure the Kafka-Spark integration by configuring SSL/TLS encryption and SASL authentication if required by your Kafka setup.
- **Performance Tuning:** Optimize the performance of your Kafka streaming application by tuning Spark configurations, such as `spark.sql.shuffle.partitions`, and Kafka consumer properties.

```
from pyspark.sql.functions import from_json, col

kafka_df = spark.readStream.format("kafka").option("kafka.bootstrap.servers",
"localhost:9092").option("subscribe", "topic").load()
kafka_df = kafka_df.selectExpr("CAST(key AS STRING)", "CAST(value AS STRING)")

schema = "id INT, value STRING"
json_df = kafka_df.select(from_json(col("value"),
schema).alias("data")).select("data.*")

query =
json_df.writeStream.outputMode("append").format("console").option("checkpointLocation",
"/path/to/checkpoint/dir").start()
query.awaitTermination()
```

Applying Window Aggregations to Streaming Data with Apache Spark Structured Streaming

- **Time-based Windows:** Define time-based windows to aggregate streaming data over fixed intervals. Use `window` functions to specify the window duration and slide interval.
- **Session Windows:** Use session windows to aggregate data based on sessions of activity. This is useful for user activity tracking where the session duration is dynamic.
- **Watermarking for Late Data:** Apply watermarks to handle late-arriving data within a specified window. This ensures that the aggregation results are complete and accurate.
- **Grouped Aggregations:** Combine window functions with `groupBy` operations to perform grouped aggregations on streaming data. This is useful for generating insights based on grouped time intervals.

- **Custom Aggregations:** Implement custom aggregation functions to perform complex computations on streaming data. Use user-defined functions (UDFs) if necessary.
- **Stateful Aggregations:** Manage stateful aggregations to keep track of running totals or averages across windows. Ensure that the state is efficiently managed to prevent memory issues.
- **Performance Optimization:** Tune window aggregation performance by adjusting window sizes, slide intervals, and Spark configurations to balance processing speed and resource usage.

```
from pyspark.sql.functions import window

windowed_counts = json_df.groupBy(window(col("timestamp"), "10 minutes")).count()

query =
windowed_counts.writeStream.outputMode("update").format("console").option("checkpointLocation", "/path/to/checkpoint/dir").start()
query.awaitTermination()
```

Handling Out-of-Order and Late-Arriving Events with Watermarking in Apache Spark Structured Streaming

- **Define Watermarks:** Specify watermarks to manage late-arriving data. Use the `withWatermark` function to set the watermark delay, allowing late data to be included in computations within a certain time frame.
- **Event Time vs. Processing Time:** Understand the difference between event time and processing time. Watermarks operate on event time to handle late data based on when the event actually occurred.
- **Window Aggregations with Watermarks:** Combine watermarks with window aggregations to handle late data effectively. This ensures accurate results even when data arrives out of order.

- **State Management:** Manage the state of streaming queries with watermarks to ensure that old data is efficiently cleaned up, preventing memory overflow and ensuring high performance.
- **Handling Extreme Delays:** Configure strategies to handle extremely delayed data, such as redirecting it to a separate processing pipeline or logging it for further analysis.
- **Monitoring Watermarks:** Continuously monitor watermark progress and late data handling to ensure that your streaming application performs as expected.
- **Error Handling:** Implement robust error handling to manage scenarios where data arrives much later than expected, ensuring that the streaming application can gracefully handle such cases.

```
watermarked_df = json_df.withWatermark("timestamp", "10 minutes")

windowed_counts = watermarked_df.groupBy(window(col("timestamp"), "10
minutes")).count()

query =
windowed_counts.writeStream.outputMode("update").format("console").option("checkpointLocation", "/path/to/checkpoint/dir").start()
query.awaitTermination()
```

Configuring Checkpoints for Structured Streaming in Apache Spark

- **Purpose of Checkpoints:** Understand the importance of checkpoints in maintaining state and fault tolerance for streaming applications. Checkpoints store progress information to recover from failures.
- **Checkpoint Configuration:** Configure the `checkpointLocation` option in your streaming query to specify where Spark should store checkpoint data. This is crucial for stateful operations and exactly-once semantics.

- **Incremental Checkpointing:** Utilize incremental checkpointing to only store changes since the last checkpoint, reducing storage overhead and improving recovery times.
- **Durable Storage for Checkpoints:** Ensure that checkpoint data is stored in a durable, reliable storage system such as HDFS, S3, or a distributed file system to prevent data loss.
- **Managing Checkpoint Data:** Regularly clean up old checkpoint data to prevent storage bloat. Implement policies to manage the lifecycle of checkpoint data.
- **Checkpoint Compatibility:** Ensure that checkpoint data is compatible with the version of Spark being used. Be cautious when upgrading Spark versions as checkpoint formats may change.
- **Monitoring Checkpoints:** Continuously monitor checkpoint progress and health to ensure that your streaming application maintains high availability and fault tolerance.

```
query = json_df.writeStream.outputMode("append").format("parquet").option("path",  
"/path/to/output/dir").option("checkpointLocation",  
"/path/to/checkpoint/dir").start()  
query.awaitTermination()
```

Processing Streaming Data

In this section, let us learn different aspects and techniques involved in processing the streaming data.

Writing the Output of Apache Spark Structured Streaming to a Sink such as Delta Lake

- **Delta Lake Integration:** Utilize Delta Lake as a sink for structured streaming data to benefit from ACID transactions and scalable metadata handling. Configure the output format as `delta`.

- **Schema Enforcement:** Leverage Delta Lake's schema enforcement to ensure data consistency and integrity. This prevents corrupted data from being written to the sink.
- **Partitioning Data:** Partition the output data based on specific columns to optimize query performance and manage large datasets efficiently. Delta Lake supports dynamic partitioning.
- **Merge Operations:** Use Delta Lake's `merge` operations to upsert data efficiently. This is useful for handling late-arriving data or implementing Change Data Capture (CDC) patterns.
- **Optimizing Delta Tables:** Optimize Delta tables using `OPTIMIZE` and `VACUUM` commands to compact files and remove old data, improving performance and reducing storage costs.
- **Checkpointing and Fault Tolerance:** Configure checkpointing to ensure fault tolerance and state management. This is essential for maintaining exactly-once semantics in streaming applications.
- **Monitoring and Metrics:** Monitor the performance and health of Delta Lake streaming applications using Spark's built-in metrics and logging capabilities.

```
delta_streaming_query =  
json_df.writeStream.format("delta").outputMode("append").option("checkpointLocation"  
, "/path/to/checkpoint/dir").option("path", "/path/to/delta/table").start()  
delta_streaming_query.awaitTermination()
```

Joining Streaming Data with Static Data in Apache Spark Structured Streaming and Delta Lake

- **Static Data as Broadcast Variables:** Use broadcast variables to efficiently join streaming data with static data. This minimizes data shuffling and improves join performance.

- **Delta Lake for Static Data:** Store static data in Delta Lake tables to leverage Delta Lake's optimization features and seamless integration with Spark Structured Streaming.
- **Efficient Join Strategies:** Choose appropriate join strategies based on the size of the data sets. Broadcast joins are ideal for small static data, while shuffle joins are necessary for larger datasets.
- **Schema Management:** Ensure that the schemas of the streaming and static data are compatible for join operations. Use Spark's schema evolution features if necessary.
- **Handling Late Data:** Manage late-arriving data by implementing watermarking and join window configurations. This ensures that all relevant data is included in the join results.
- **Performance Tuning:** Optimize join performance by tuning Spark configurations such as `spark.sql.autoBroadcastJoinThreshold` and `spark.sql.shuffle.partitions`.
- **Monitoring and Debugging:** Monitor the join operations using Spark UI and logs to identify and resolve performance bottlenecks or data issues.

```
static_df = spark.read.format("delta").load("/path/to/static/data")
joined_df = streaming_df.join(static_df, "id")

query =
  joined_df.writeStream.format("console").outputMode("append").option("checkpointLocation", "/path/to/checkpoint/dir").start()
query.awaitTermination()
```

Joining Streaming Data with Static Data in Apache Spark Structured Streaming and Delta Lake

- **Stream-to-Stream Joins:** Enable stream-to-stream joins to combine data from two streaming sources. This is useful for scenarios where insights are derived from multiple real-time data streams.

- **Watermarking for Both Streams:** Apply watermarks to both streaming datasets to manage late data and ensure accurate join results. Watermarks help in maintaining the state within a manageable size.
- **Window-based Joins:** Use window-based joins to restrict the join operation to specific time intervals. This reduces the computational complexity and ensures timely processing of data.
- **Handling State and Memory:** Manage the state and memory usage effectively by configuring appropriate state timeout settings. This helps prevent memory overflow and ensures smooth operation.
- **Join Conditions:** Define clear join conditions to ensure that only relevant data is combined. This includes specifying the join keys and any additional filters.
- **Monitoring Performance:** Continuously monitor the performance of stream-to-stream joins using Spark metrics and logs. Identify and address any performance bottlenecks or data issues.
- **Error Handling and Recovery:** Implement robust error handling and recovery mechanisms to manage failures in stream-to-stream joins. This ensures the resilience and reliability of the streaming application.

```
streaming_df1 = spark.readStream.format("kafka").option("kafka.bootstrap.servers",
"localhost:9092").option("subscribe", "topic1").load()
streaming_df2 = spark.readStream.format("kafka").option("kafka.bootstrap.servers",
"localhost:9092").option("subscribe", "topic2").load()

joined_df = streaming_df1.join(streaming_df2, expr("streaming_df1.id =
streaming_df2.id AND streaming_df1.timestamp BETWEEN streaming_df2.timestamp -
interval 10 minutes AND streaming_df2.timestamp + interval 10 minutes"))

query =
joined_df.writeStream.format("console").outputMode("append").option("checkpointLocat
ion", "/path/to/checkpoint/dir").start()
query.awaitTermination()
```

Idempotent Stream Writing with Delta Lake and Apache Spark Structured Streaming

- **Idempotency in Streaming:** Ensure idempotent writes to prevent duplicate data entries in the Delta Lake. This is crucial for maintaining data accuracy and consistency in streaming applications.
- **Delta Lake Upserts:** Use Delta Lake's `merge` operation to perform upserts, ensuring that existing records are updated, and new records are inserted without duplication.
- **Unique Identifiers:** Implement unique identifiers for records to facilitate idempotent writes. Use these identifiers to match records during upsert operations.
- **Transactional Guarantees:** Leverage Delta Lake's ACID transaction capabilities to ensure that stream writes are atomic, consistent, isolated, and durable.
- **Conflict Resolution:** Define conflict resolution strategies for handling cases where multiple records with the same identifier are processed simultaneously. This ensures data integrity.
- **State Management:** Maintain the state of processed records to identify and discard duplicates efficiently. Use checkpoints and state stores to manage this state.
- **Performance Considerations:** Optimize the performance of idempotent writes by tuning Spark and Delta Lake configurations. This includes configuring write parallelism and managing transaction log size.

```

from delta.tables import *

delta_table = DeltaTable.forPath(spark, "/path/to/delta/table")

def upsert_to_delta(microBatchOutputDF, batchId):
    delta_table.alias("tgt").merge(microBatchOutputDF.alias("src"), "tgt.id =
src.id").whenMatchedUpdateAll().whenNotMatchedInsertAll().execute()

query =
json_df.writeStream.format("delta").foreachBatch(upsert_to_delta).option("checkpoint
Location", "/path/to/checkpoint/dir").start()
query.awaitTermination()

```

Merging or Applying Change Data Capture on Apache Spark Structured Streaming and Delta Lake

- **Change Data Capture (CDC):** Implement CDC to capture and apply changes in real-time, ensuring that the Delta Lake reflects the most recent state of the data. This is useful for data synchronization and replication.
- **Delta Lake Merge Operations:** Use Delta Lake's `merge` operation to apply changes captured by CDC. This ensures that updates, inserts, and deletes are correctly applied to the Delta Lake.
- **Streaming Sources for CDC:** Configure streaming sources like Kafka or database change logs to capture real-time changes. Ensure that these sources provide reliable change data streams.
- **Schema Evolution:** Manage schema evolution in CDC scenarios to accommodate changes in the data structure. Delta Lake supports schema evolution, ensuring compatibility with evolving data schemas.
- **Conflict Handling:** Implement strategies to handle conflicts that arise during the application of changes. This includes defining rules for resolving update conflicts and managing concurrent modifications.
- **Performance Optimization:** Optimize CDC performance by tuning Spark configurations and Delta Lake settings. This includes managing transaction log size and configuring appropriate partitioning strategies.

- **Monitoring and Auditing:** Continuously monitor CDC processes to ensure that changes are applied accurately and efficiently. Implement auditing mechanisms to track the history of changes applied to the Delta Lake.

```
cdc_df = spark.readStream.format("kafka").option("kafka.bootstrap.servers",
"localhost:9092").option("subscribe", "cdc_topic").load()
cdc_df = cdc_df.selectExpr("CAST(key AS STRING)", "CAST(value AS STRING)")

def apply_cdc(microBatchOutputDF, batchId):
    delta_table.alias("tgt").merge(microBatchOutputDF.alias("src"), "tgt.id =
src.id").whenMatchedUpdateAll().whenNotMatchedInsertAll().execute()

cdc_query =
cdc_df.writeStream.format("delta").foreachBatch(apply_cdc).option("checkpointLocatio
n", "/path/to/checkpoint/dir").start()
cdc_query.awaitTermination()
```

Monitoring Real-time Data Processing with Apache Spark Structured Streaming

- **Spark UI:** Utilize the Spark UI to monitor real-time data processing. The UI provides detailed insights into the performance and health of streaming queries.
- **Structured Streaming Metrics:** Access built-in metrics for structured streaming, such as input rates, processing rates, and latency. Use these metrics to track the performance of your streaming application.
- **Custom Monitoring:** Implement custom monitoring solutions using Spark's metrics system. Publish metrics to external systems like Prometheus or Grafana for advanced monitoring and alerting.
- **Logging:** Configure logging to capture detailed information about streaming query execution. Use log aggregation tools to analyze logs and troubleshoot issues.
- **Alerts and Notifications:** Set up alerts and notifications to receive real-time updates about the status of your streaming application. This helps in proactive management and quick resolution of issues.

- **Checkpoint Monitoring:** Monitor the health and progress of checkpoints to ensure that your streaming application maintains fault tolerance and state management.
- **Resource Utilization:** Track the resource utilization of your streaming application, including CPU, memory, and network usage. Optimize resource allocation to improve performance and reduce costs.

```
# Example code to set up custom metrics in Spark Structured Streaming
spark.conf.set("spark.metrics.namespace", "structured_streaming_example")
spark.conf.set("spark.metrics.conf", "/path/to/metrics/conf/file")

# Example configuration in the metrics conf file
# [*.source.jvm]
# class=org.apache.spark.metrics.source.JvmSource

query =
  json df.writeStream.format("delta").outputMode("append").option("checkpointLocation"
, "/path/to/checkpoint/dir").start()
query.awaitTermination()
```

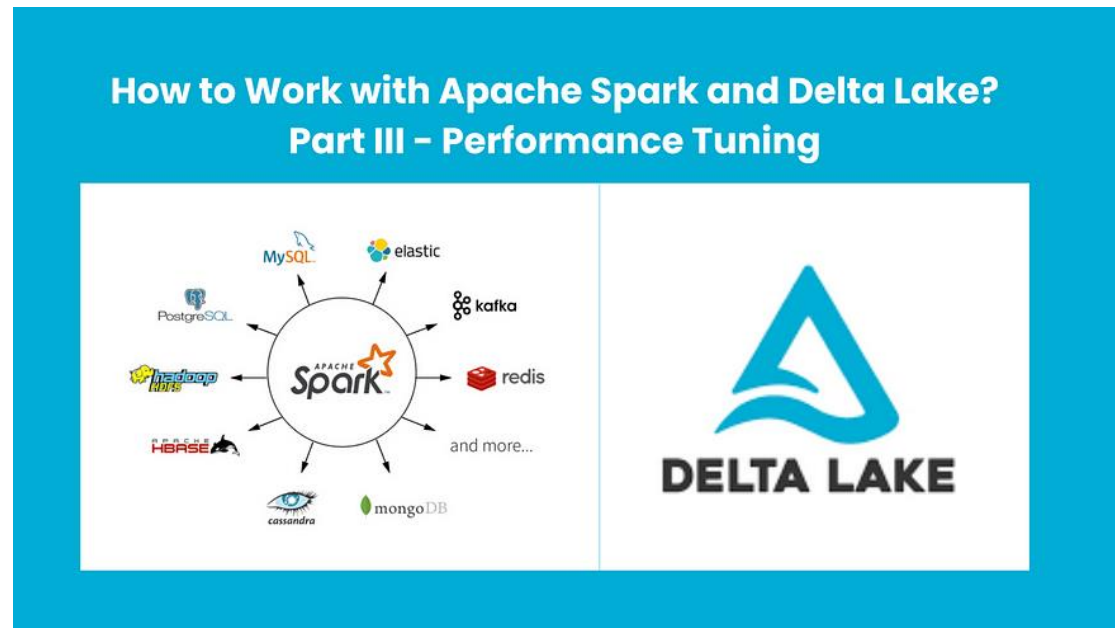
What's Next?

In the next section, we will explore the following topics:

- **Performance Tuning:** Strategies for optimizing Spark applications, including resource allocation, partitioning, and shuffling. Use Spark UI for performance monitoring.
- **Performance Tuning in Delta Lake:** Z-Ordering, Skipping Data and I/O optimization.

How to Work with Apache Spark and Delta Lake? — Performance Tuning

Learn the methods to tune the performance of Apache Spark Streaming and Delta Lake.



Performance Tuning of Streaming Data

Here is a guide to tune performance for your streaming data:

Monitoring Spark Jobs

- **Spark UI:** Utilize the Spark UI to monitor the execution of Spark jobs. The UI provides a detailed view of job stages, tasks, and execution timelines.
- **Job Metrics:** Access job metrics such as task duration, shuffle read/write, and executor utilization. Use these metrics to identify performance bottlenecks and optimize job execution.
- **Event Logs:** Analyze Spark event logs to gain insights into job execution. Event logs capture detailed information about job stages, tasks, and resource usage.

- **Third-Party Monitoring Tools:** Integrate third-party monitoring tools like Ganglia, Prometheus, or Datadog to gain advanced monitoring capabilities and real-time alerts.
- **Custom Metrics:** Implement custom metrics using Spark's metrics system. Publish these metrics to monitoring dashboards for continuous tracking.
- **Cluster Management:** Monitor the health and utilization of your Spark cluster using cluster management tools like YARN, Mesos, or Kubernetes.
- **Performance Dashboards:** Create performance dashboards to visualize key metrics and trends. Use these dashboards to make data-driven decisions for optimizing Spark jobs.

```
# Example code to access Spark job metrics
from pyspark.sql.functions import col

job_metrics = spark.sql("SHOW STAGES FROM job_id")
job_metrics.show()

# Example code to set up custom metrics
spark.conf.set("spark.metrics.namespace", "performance_tuning_example")
spark.conf.set("spark.metrics.conf", "/path/to/metrics/conf/file")
```

Using Broadcast Variables

- **Broadcast Variables for Efficiency:** Use broadcast variables to efficiently distribute large read-only data across all nodes. This minimizes data shuffling and improves query performance.
- **Creating Broadcast Variables:** Create broadcast variables using the `sparkContext.broadcast` method. Access the broadcasted data using the `value` attribute.
- **Optimizing Joins:** Utilize broadcast variables for small tables in join operations. This ensures that the small table is distributed to all nodes, reducing the need for data shuffling.

- **Caching Broadcast Data:** Cache broadcasted data in memory to reduce the overhead of repeatedly broadcasting the same data. This is useful for iterative algorithms.
- **Memory Management:** Monitor and manage the memory usage of broadcast variables to prevent memory overflow. Adjust Spark configurations if necessary.
- **Broadcast Variable Updates:** Update broadcast variables only when necessary. Frequent updates can lead to performance degradation due to the overhead of re-broadcasting data.
- **Fault Tolerance:** Ensure fault tolerance by managing the state of broadcast variables. Spark handles the recovery of broadcast variables in case of node failures.

```
# Example code to create and use broadcast variables
broadcast_data = spark.sparkContext.broadcast([1, 2, 3, 4, 5])

df = spark.createDataFrame([(1, "A"), (2, "B"), (3, "C")], ["id", "value"])
result_df = df.filter(col("id").isin(broadcast_data.value))

result_df.show()
```

Avoiding Data Skew

- **Identifying Data Skew:** Monitor job execution and identify stages with long-running tasks. Data skew occurs when certain partitions contain significantly more data than others.
- **Salting Keys:** Implement key salting to distribute skewed data more evenly across partitions. Append a random value to the keys to break up large groups of data.
- **Custom Partitioning:** Define custom partitioning strategies to balance the data distribution. Use Spark's `partitionBy` method to specify the partitioning columns.

- **Combining Small Partitions:** Combine small partitions to reduce overhead and improve parallelism. Use `coalesce` to merge small partitions without a full shuffle.
- **Repartitioning Data:** Repartition data to achieve an optimal number of partitions. Use `repartition` to distribute data evenly across all nodes.
- **Adaptive Query Execution:** Enable Spark's Adaptive Query Execution (AQE) to dynamically optimize query plans and manage data skew. AQE adjusts the number of partitions based on runtime statistics.
- **Monitoring and Tuning:** Continuously monitor the performance of your Spark jobs and tune configurations to address data skew issues. Use metrics and logs to identify skew patterns.

```
# Example code to implement key salting
from pyspark.sql.functions import concat, lit

salted_df = df.withColumn("salted_key", concat(col("key"), lit("_"), lit(rand())))
salted_df = salted_df.repartition("salted_key")

# Example code to use Adaptive Query Execution
spark.conf.set("spark.sql.adaptive.enabled", "true")
result_df = df.groupBy("key").count()
result_df.show()
```

Caching and Shuffling

- **Caching DataFrames:** Cache frequently accessed DataFrames to memory to speed up query execution. Use `cache` or `persist` methods to cache data.
- **Memory Management:** Monitor and manage memory usage when caching data. Use Spark's storage tab in the UI to track cached DataFrames and their memory consumption.
- **Efficient Use of Cache:** Use caching judiciously for intermediate results that are reused multiple times. Avoid caching large DataFrames that are used only once.

- **Clearing Cache:** Clear the cache when it is no longer needed to free up memory. Use the `unpersist` method to remove cached DataFrames.
- **Shuffling Optimization:** Optimize shuffling by adjusting Spark configurations such as `spark.sql.shuffle.partitions`. Properly configure the number of partitions to balance shuffle load.
- **Avoiding Unnecessary Shuffles:** Minimize the number of shuffle operations by using appropriate transformations. Avoid transformations that trigger shuffles unless necessary.
- **Broadcast Joins:** Use broadcast joins to reduce shuffle operations. Broadcast small tables to all nodes to perform join operations locally.

```
# Example code to cache a DataFrame
cached_df = df.cache()
result_df = cached_df.groupBy("key").count()
result_df.show()

# Example code to unpersist a DataFrame
cached_df.unpersist()

# Example code to set shuffle partitions
spark.conf.set("spark.sql.shuffle.partitions", "200")
```

Partitioning and Repartitioning

- **Initial Partitioning:** Ensure that data is properly partitioned at the source to avoid the need for extensive repartitioning. Use appropriate partitioning columns to distribute data evenly.
- **Repartitioning Data:** Use the `repartition` method to increase or decrease the number of partitions. This helps in balancing the load across all nodes.
- **Coalescing Partitions:** Use the `coalesce` method to reduce the number of partitions without a full shuffle. This is useful for optimizing data storage and query performance.
- **Partitioning for Joins:** Optimize joins by partitioning data on the join keys. This reduces data shuffling and improves the efficiency of join operations.

- **Skewed Data Management:** Address skewed data by implementing custom partitioning strategies. Use key salting or custom hash functions to distribute data evenly.
- **Dynamic Partitioning:** Enable dynamic partitioning to adjust the number of partitions based on runtime statistics. Spark's Adaptive Query Execution (AQE) can automatically optimize partitioning.
- **Monitoring and Tuning:** Continuously monitor the performance of your Spark jobs and adjust partitioning strategies as needed. Use metrics and logs to identify partitioning issues.

```
# Example code to repartition a DataFrame
repartitioned_df = df.repartition(100, "key")
result_df = repartitioned_df.groupBy("key").count()
result_df.show()

# Example code to coalesce a DataFrame
coalesced_df = df.coalesce(50)
result_df = coalesced_df.groupBy("key").count()
result_df.show()
```

Optimizing Join Strategies

- **Broadcast Joins:** Use broadcast joins for small tables to minimize data shuffling. Broadcast the small table to all nodes to perform join operations locally.
- **Sort-Merge Joins:** Utilize sort-merge joins for large tables. Ensure that the join keys are sorted to optimize the merge operation.
- **Shuffle Hash Joins:** Implement shuffle hash joins for medium-sized tables. Partition the tables on the join keys to distribute the data evenly.
- **Skewed Data Handling:** Address skewed data by using key salting or custom partitioning strategies. This helps in balancing the load across all nodes during join operations.

- **Join Hints:** Use join hints to guide Spark in selecting the optimal join strategy. Hints like `broadcast`, `merge`, and `shuffle` can improve join performance.
- **Adaptive Query Execution:** Enable Adaptive Query Execution (AQE) to dynamically optimize join strategies based on runtime statistics. AQE adjusts the join strategy based on the size of the data.
- **Monitoring and Tuning:** Continuously monitor the performance of join operations and adjust strategies as needed. Use metrics and logs to identify and resolve join performance issues.

```
# Example code to use a broadcast join
from pyspark.sql.functions import broadcast

small_df = spark.read.format("delta").load("/path/to/small/table")
large_df = spark.read.format("delta").load("/path/to/large/table")

joined_df = large_df.join(broadcast(small_df), "key")
joined_df.show()

# Example code to enable Adaptive Query Execution
spark.conf.set("spark.sql.adaptive.enabled", "true")
result_df = df.groupBy("key").count()
result_df.show()
```

Performance Tuning in Delta Lake

Here is how you can tune the performance in Delta Lake:

Optimizing Delta Lake for Performance

- **Schema Optimization:** Design an efficient schema with proper data types to optimize Delta Lake performance. Use compact data types and avoid nested structures when possible.
- **Partitioning Strategy:** Implement an optimal partitioning strategy to balance the load and improve query performance. Use columns with high cardinality for partitioning.

- **Compaction:** Regularly compact small files into larger ones using the `OPTIMIZE` command. This reduces the number of files and improves read performance.
- **Z-Ordering:** Apply Z-order clustering to sort data within partitions based on query patterns. This improves data skipping and reduces scan times.
- **Caching:** Cache frequently accessed data in memory to speed up query execution. Use `cache` or `persist` methods to cache Delta tables.
- **Data Skipping:** Enable data skipping to avoid scanning unnecessary data. Delta Lake automatically maintains statistics for efficient data skipping.
- **Vacuuming:** Use the `VACUUM` command to remove old versions of data and free up storage space. This helps in managing storage costs and improving performance.

```
# Example code to optimize a Delta table
delta_table = DeltaTable.forPath(spark, "/path/to/delta/table")
delta_table.optimize().executeCompaction()

# Example code to apply Z-order clustering
delta_table.optimize().zorderBy("key").executeCompaction()

# Example code to vacuum a Delta table
delta_table.vacuum(7) # Retain data from the last 7 days
```

Z-Order for Query Execution

- **Definition of Z-Order:** Understand that Z-ordering is a technique to sort data within partitions based on multiple columns. This improves data skipping and query performance.
- **Applying Z-Order:** Use the `OPTIMIZE` command with `ZORDER BY` to apply Z-order clustering to Delta tables. Choose columns frequently used in filter conditions.
- **Benefits of Z-Order:** Z-ordering reduces the amount of data read during query execution, leading to faster query performance. It is especially beneficial for large tables.

- **Choosing Z-Order Columns:** Select columns with high cardinality and frequent use in queries for Z-ordering. This maximizes the benefits of data skipping.
- **Monitoring Z-Order Performance:** Continuously monitor query performance and adjust Z-order columns as needed. Use query metrics and logs to evaluate the impact of Z-ordering.
- **Combining with Partitioning:** Combine Z-ordering with partitioning to optimize query performance further. Partition on high cardinality columns and Z-order on columns frequently used in filters.
- **Example Use Cases:** Use Z-ordering for time-series data, event logs, or any scenario where queries often filter on multiple columns.

```
# Example code to apply Z-order clustering
delta table = DeltaTable.forPath(spark, "/path/to/delta/table")
delta_table.optimize().zorderBy("timestamp", "user_id").executeCompaction()
```

Skipping Data

- **Definition of Data Skipping:** Data skipping is a technique to avoid scanning irrelevant data based on stored statistics. Delta Lake maintains min/max values for efficient data skipping.
- **Enabling Data Skipping:** Data skipping is automatically enabled in Delta Lake. Ensure that your queries utilize filters to benefit from data skipping.
- **Monitoring Data Skipping:** Monitor query plans and execution to verify that data skipping is effectively reducing the amount of data read. Use Spark UI and query metrics for analysis.
- **Optimizing Skipping Performance:** Optimize data skipping performance by partitioning and Z-ordering data. These techniques improve the granularity of statistics and enhance skipping efficiency.

- **Use Cases for Data Skipping:** Data skipping is beneficial for scenarios with large datasets and queries with selective filters. Examples include log analysis, time-series data, and user activity tracking.
- **Combining Techniques:** Combine data skipping with other optimization techniques like partitioning, Z-ordering, and caching to maximize query performance.
- **Example Queries:** Design queries that leverage data skipping by including filter conditions on indexed columns. This reduces the amount of data scanned and speeds up query execution.

```
# Example query leveraging data skipping
filtered_df = delta_table.filter("timestamp >= '2024-01-01' AND user_id = '12345'")
result_df = filtered_df.groupBy("event_type").count()
result_df.show()
```

Reducing Delta Lake Table Size and I/O Cost

- **File Compaction:** Regularly compact small files into larger ones using the `OPTIMIZE` command. This reduces the number of files and improves read performance.
- **Data Compression:** Use efficient data compression formats like Parquet or ORC to reduce storage space and I/O costs. Delta Lake supports these formats natively.
- **Column Pruning:** Ensure that queries only read the necessary columns by leveraging column pruning. This reduces the amount of data read and improves query performance.
- **Partition Pruning:** Implement partition pruning to read only the relevant partitions based on query filters. This minimizes the amount of data scanned and reduces I/O costs.
- **Caching Hot Data:** Cache frequently accessed data in memory to reduce I/O operations. Use the `cache` or `persist` methods to cache Delta tables.

- **Schema Design:** Design an efficient schema with compact data types to reduce storage space. Avoid nested structures and use appropriate data types.
- **Vacuuming Old Data:** Use the `VACUUM` command to remove old versions of data and free up storage space. This helps in managing storage costs and improving performance.

```
# Example code to optimize a Delta table and reduce file size
delta_table = DeltaTable.forPath(spark, "/path/to/delta/table")
delta_table.optimize().executeCompaction()

# Example code to vacuum a Delta table
delta_table.vacuum(7) # Retain data from the last 7 days

# Example query leveraging column pruning
result_df = delta_table.select("user_id", "event_type").filter("timestamp >= '2024-01-01'")
result_df.show()
```

Wrapping Up

In these 3 parts blog series, you would have learnt about:

- **Data Operations:** Data Ingestion, Data Extraction, Data Management & and manipulation in Delta Lake.
- **Streaming Data:** Handle real-time data streams with Apache Spark. Set up data streams from sources like Kafka, socket connections, and files.
- **Processing Streaming Data:** Techniques for processing streaming data, including windowed operations and stateful processing. Manage watermarks and late data.
- **Performance Tuning:** Strategies for optimizing Spark applications, including resource allocation, partitioning, and shuffling. Use Spark UI for performance monitoring.
- **Performance Tuning in Delta Lake:** Z-Ordering, Skipping Data and I/O optimization.

Reference: <https://github.com/PacktPublishing/Data-Engineeringwith-Databricks-Cookbook.git>